

Adding Nested JSON Objects to Azure Cosmos DB using .NET SDK

Summary:-

In this lab, you will modify the .NET application to insert JSON documents containing nested objects into an Azure Cosmos DB container. You will create separate classes for course and payment information, embed them within the Order class, and verify that the nested JSON documents are stored successfully.

Let's Start:-

After completing this lab, you will be able to:

- Create nested objects in a .NET model.
- Embed JSON objects inside a JSON document.
- Insert nested JSON documents into Azure Cosmos DB.
- Verify nested objects using Data Explorer.

Lab Steps:-

Step 1 – Prepare the Azure Cosmos DB .NET Project

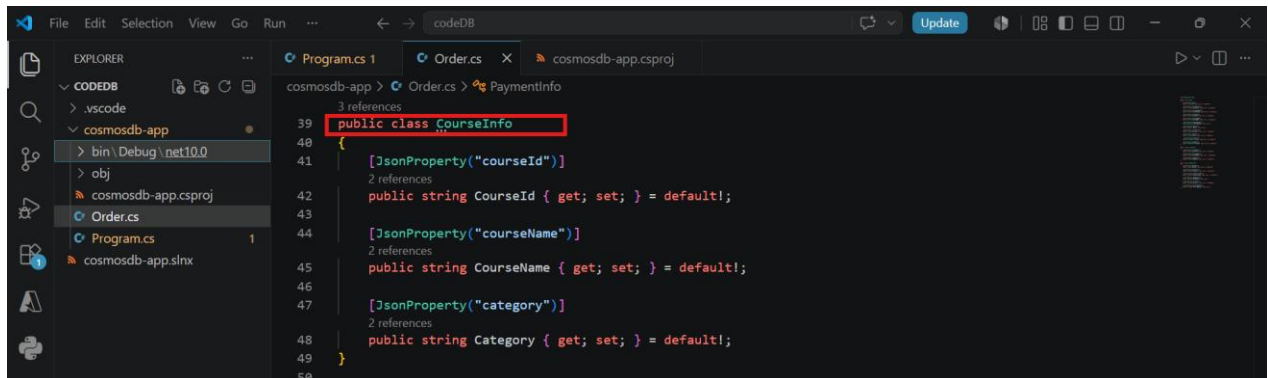
- **Reference:** This lab uses the .NET console application, Azure Cosmos DB account, database, and container created in the previous lab.
- For that **create** Azure Cosmos DB account by selecting **Azure Cosmos DB for NoSQL** from the recommended APIs.
- Configure:
Workload type: Learning, **Region:** EAST US/ EAST US 2, **Capacity Mode:** Provisioned Throughput
- Enable **Apply Free Tier Discount**. Click **Next**.
- Ensure Geo-Redundancy and Multi-region Writes is **Disabled**. Click **Next**.
- Select **Enable access from all networks**. Click on **Review + Create** to create.
- From the left pane, select **Keys**.
- Copy: **URI** and **Primary Key**
- Open **Visual Studio Code**. Create a new project folder.
- Press **Ctrl + Shift + P** to open the **Command Palette**.
- Select **Show and Run Commands > .NET: New Project > Console App**(Rename it as cosmosdb-app) > **Default Directory > .slnx > Create Project**.
- **Open** the integrated terminal. **Run:**

```
cd .\cosmosdb-app\  
dotnet add package Microsoft.Azure.Cosmos  
dotnet add package Newtonsoft.Json
```
- Create a new class file named **Order.cs**.

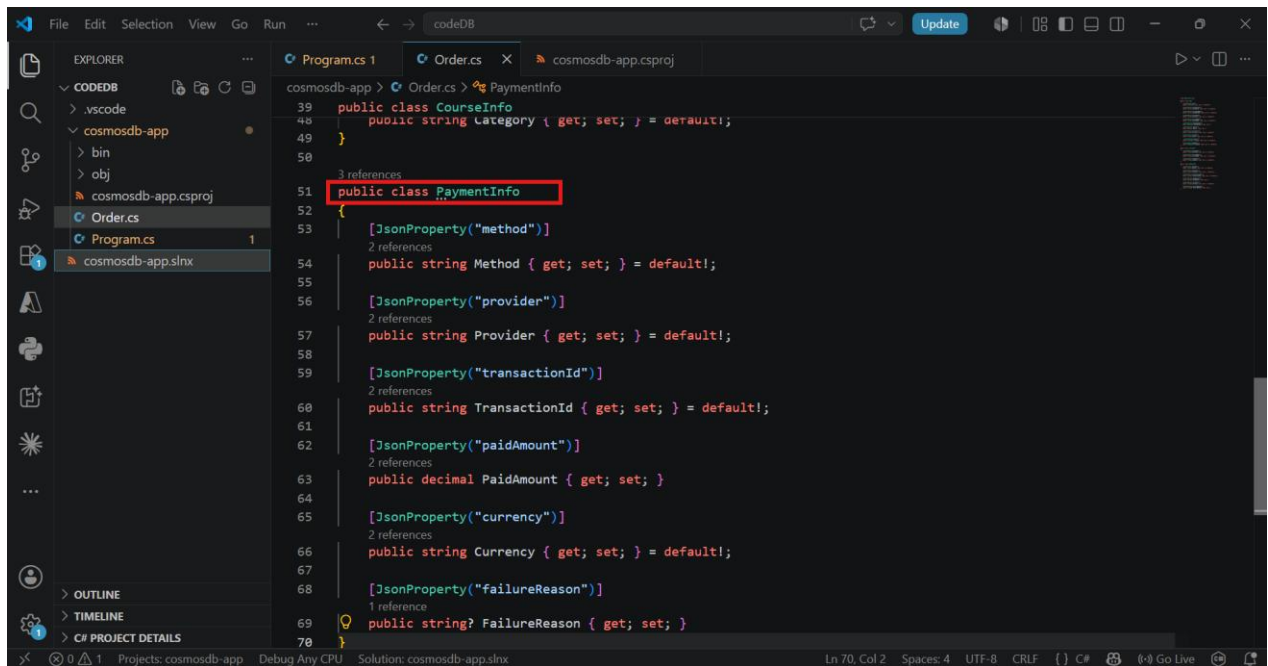
- Open the following project files:
Program.cs
Order.cs
cosmosdb-app.csproj
- Replace the existing content with the **provided files**.
- In **Program.cs**, **Replace** Endpoint with **URI** and Key with **Primary Key**.
- **Save** all the files. **Run**:
dotnet run
dotnet build
- This step creates a .NET console application and prepares it to insert items into the Azure Cosmos DB container.

Step 2 – Update the Model Classes

- Open **Order.cs** and replace the existing code with the provided **Order.cs** file.
- Ensure the file includes:
 - **CourseInfo** class



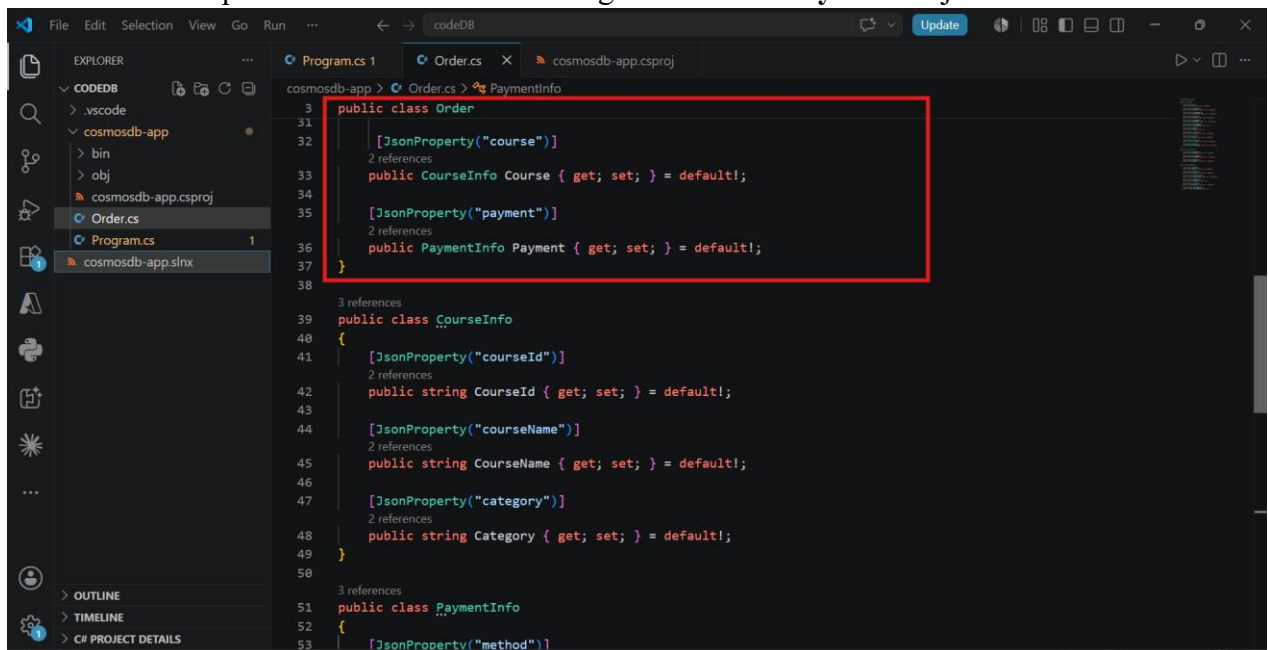
- **PaymentInfo** class



The screenshot shows the Visual Studio Code editor with the `Order.cs` file open. The `PaymentInfo` class is highlighted with a red box. The code defines the `PaymentInfo` class with properties for `Method`, `Provider`, `TransactionId`, `PaidAmount`, `Currency`, and `FailureReason`. The `FailureReason` property is nullable (`string?`).

```
39 public class CourseInfo
40 {
41     public string Category { get; set; } = default!;
42 }
43
44 3 references
45 public class PaymentInfo
46 {
47     [JsonProperty("method")]
48     public string Method { get; set; } = default!;
49
50     [JsonProperty("provider")]
51     public string Provider { get; set; } = default!;
52
53     [JsonProperty("transactionId")]
54     public string TransactionId { get; set; } = default!;
55
56     [JsonProperty("paidAmount")]
57     public decimal PaidAmount { get; set; }
58
59     [JsonProperty("currency")]
60     public string Currency { get; set; } = default!;
61
62     [JsonProperty("failureReason")]
63     public string? FailureReason { get; set; }
64 }
```

○ Updated **Order** class containing **Course** and **Payment** objects.



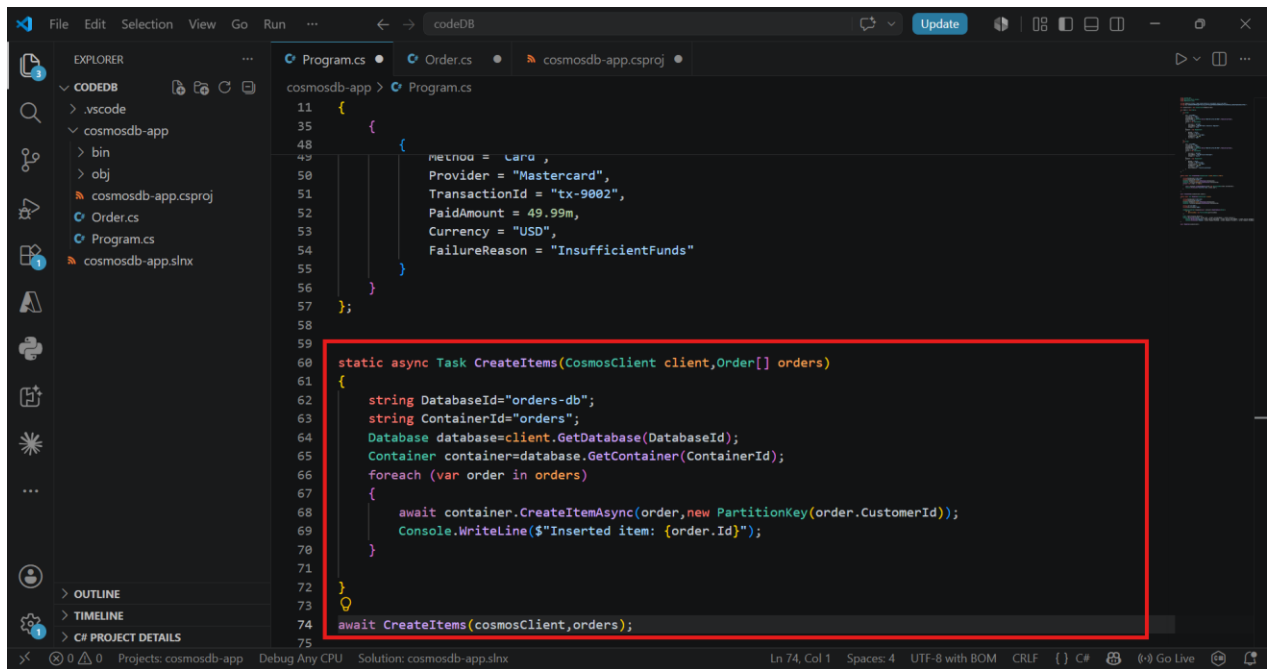
The screenshot shows the Visual Studio Code editor with the `Order.cs` file open. The `Order` class is highlighted with a red box. The code defines the `Order` class with properties for `Course` and `Payment`, both of which are objects of the `CourseInfo` and `PaymentInfo` classes respectively.

```
3 public class Order
4 {
5     [JsonProperty("course")]
6     public CourseInfo Course { get; set; } = default!;
7
8     [JsonProperty("payment")]
9     public PaymentInfo Payment { get; set; } = default!;
10 }
11
12 3 references
13 public class CourseInfo
14 {
15     [JsonProperty("courseId")]
16     public string CourseId { get; set; } = default!;
17
18     [JsonProperty("courseName")]
19     public string CourseName { get; set; } = default!;
20
21     [JsonProperty("category")]
22     public string Category { get; set; } = default!;
23 }
24
25 3 references
26 public class PaymentInfo
27 {
28     [JsonProperty("method")]
```

- Save the file. This step adds nested objects to the Order model.

Step 3 – Update the Application Code

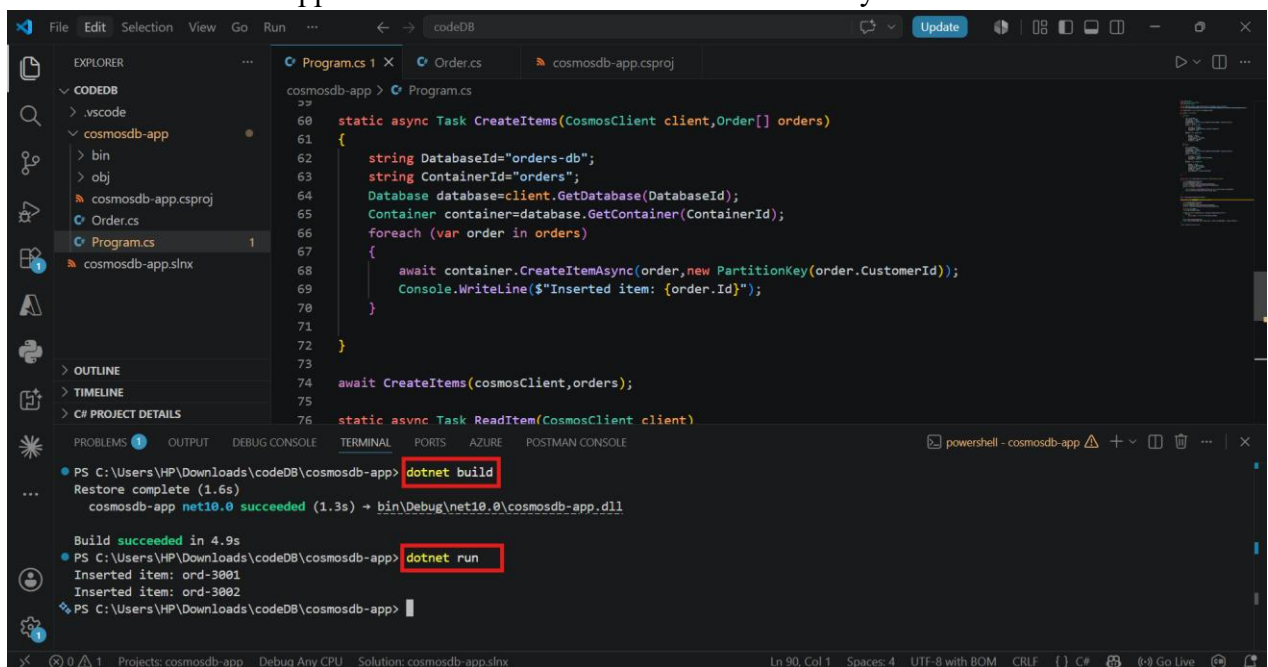
- Open **Program.cs** and replace the existing code with the provided **Program.cs** file.



- Save the file. This step updates the application to create and insert JSON documents containing nested objects.

Step 4 – Build and Run the Application

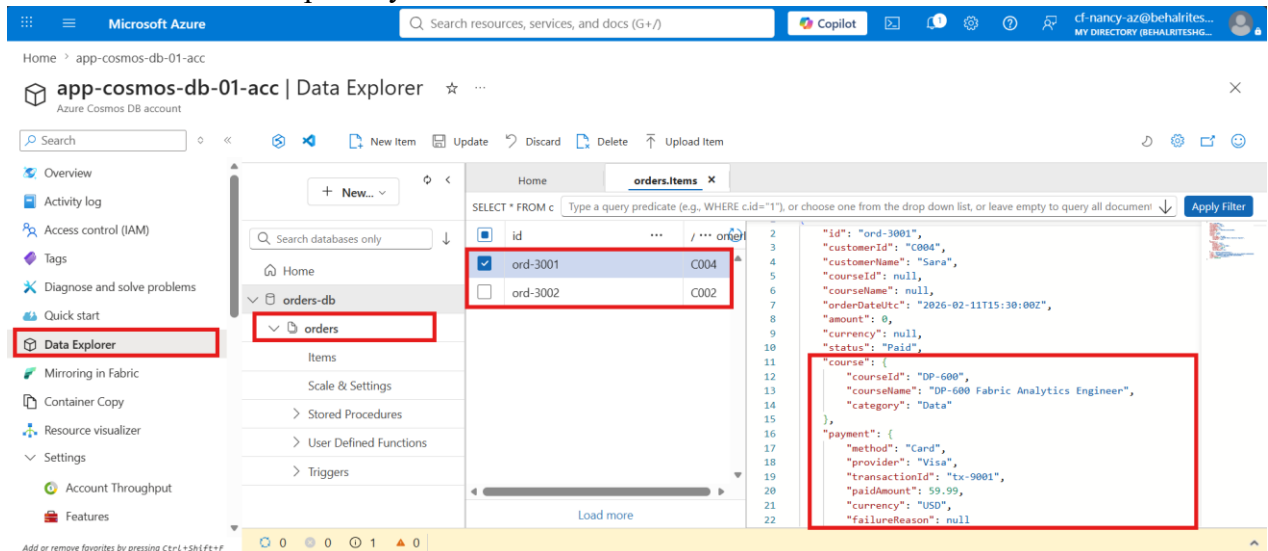
- Open the **integrated terminal** and run:
dotnet build
dotnet run
- Wait until the application inserts the documents successfully.



- This step inserts JSON documents with nested objects into the Azure Cosmos DB container.

Step 5 – Verify the Inserted Documents

- Open the Azure Portal. Navigate to **Azure Cosmos DB > Data Explorer**.
- Expand the **orders-db** database and the **orders** container.
- Select **Items**. Open any inserted document.



- Verify that:
 - The **Course** object contains the course details.
 - The **Payment** object contains the payment information.
- This step confirms that nested JSON objects are stored successfully in Azure Cosmos DB.

Verification

Verify the following:

- The application builds and runs successfully.
- Documents are inserted into the **orders** container.
- Each document contains nested **Course** and **Payment** JSON objects.
- The nested object properties are displayed correctly in **Data Explorer**.
- No errors occur during document insertion.